

# Begründung für die Implementierung: KatS-Stab-Dashboard

---

## 1. Zielgruppenanalyse

Das Dashboard richtet sich an operative Einsatzkräfte und Stabsbereiche des Katastrophenschutzes (Feuerwehr, THW, Rettungsdienste). Diese arbeiten häufig unter Zeitdruck und starker kognitiver Belastung, weshalb die schnelle, gebündelte Erfassung einsatzkritischer Lageinformationen entscheidend ist. Deshalb werden verteilte Systemdaten auf einer einzigen, übersichtlichen Oberfläche zusammengeführt. Der Fokus liegt strikt auf Performance, Ausfallsicherheit und einer klaren Informationsarchitektur, um kritische Entscheidungen ohne Verzögerung zu ermöglichen.

## 2. Technische Randbedingungen

Eine zentrale Randbedingung ist der schonende Umgang mit den Rate-Limits kostenfreier APIs. Um das System belastbar (resilient) zu machen, wurde eine Smart Polling-Architektur umgesetzt:

- **Idle-Sleep & Wakeup:** Das Backend überwacht permanent die Anzahl der verbundenen WebSocket-Clients. Sind keine Nutzer verbunden, pausiert das Polling vollständig ("Schlafmodus"), um die API-Kontingente zu schonen. Verbindet sich der erste Client, werden die Daten sofort aktualisiert.
- **Interne Cooldowns:** Service-spezifische Abklingzeiten (Wetter 10 Min., Pegel 5 Min., Einsatzdaten 1 Std.) liegen bewusst unter den Anbieterlimits, damit die zulässigen Request-Limits der Drittanbieter nicht überschritten werden.
- **Erkennung toter Verbindungen:** Verbindungen, die länger als 90 Sekunden keine Nachricht senden, werden automatisch geschlossen. Damit lösen abgehangte oder in den Ruhezustand versetzte Endgeräte keine unnötigen Abfragen mehr aus; ein Ping-Heartbeat des Browsers hält lebende Verbindungen am Leben.
- **Geschützte manuelle Refreshes:** Der WebSocket-Endpunkt ist öffentlich erreichbar, sodass jeder verbundene Client die manuellen Refresh-Buttons triggern kann. Deshalb ist ein manuelles Nachladen pro Datenquelle auf einmal alle 30 Sekunden begrenzt. Der Button zeigt dabei direkt an, ob die Aktualisierung funktioniert hat, gerade zu früh erfolgt ist oder fehlgeschlagen ist.

## 3. Entscheidungen rund um die Datenquellen

Gemäß der Anforderung, realistische und valide Live-Daten abzubilden, wurden ausschließlich offizielle öffentliche Schnittstellen angebunden – statt auf Mock-Daten zurückzugreifen:

- **Wetter (Bright Sky / DWD-Open-Data):** Die Daten des Deutschen Wetterdienstes (Messwerte der SYNOP-Stationen plus MOSMIX-Stationsvorhersage) liefern höchste lokale Präzision für Berlin: aktuelle Werte, eine stündliche Prognose (24 Stunden) und die Tagesprognose der kommenden sieben Tage (ohne den heutigen Tag). Hinzu kommen die amtlichen DWD-Warnungen der Warnzelle, deren Schweregrad auf die offiziellen Warnstufen 1–4 (gelb bis violett) abgebildet wird. Die

Entscheidung fiel gezielt auf diese Quelle, weil Open-Meteo ein Tageslimit pro IP kennt, das auf der geteilten IP des Hosters von anderen Nutzern aufgebraucht wurde – im Abnahmebetrieb lief das Dashboard dadurch regelmäßig ohne Wetterdaten. Bright Sky hat keinen API-Key und kein solches Limit, was die Tests reproduzierbar gemacht hat.

- Pegelstände (Pegelonline / WSV): Die REST-API der Wasserstraßen- und Schifffahrtsverwaltung wird genutzt, um Berliner Messstellen dynamisch abzufragen und 24-Stunden-Verläufe mit den Referenzwerten NNW bis HHW darzustellen. Die Tendenz (steigend/fallend) wird dabei nicht über eine Zwei-Punkt-Differenz, sondern als lineare Regression über einen wählbaren Zeitraum (1–24 h) berechnet, da die 15-Minuten-Messwerte um  $\pm 2$  cm schwanken und ein enges Vergleichsfenster sonst ständig das falsche Signal liefern würde.
- Einsatzdaten (Berliner Feuerwehr): Ein täglich aktualisierter CSV-Abzug aus dem Open-Data-Angebot liefert die Einsätze des Vortags. Für das farbliche Einsatzspektrum liefert eine zweite Datei (Hauptbeschwerden des Jahres) die Top-7-Meldebildkategorien, die auf die Rettungsdienstzahl des Vortags skaliert und als Treemap visualisiert werden.

Die Herkunft aller drei Quellen wird direkt an der jeweiligen Anzeige mit ihrer Lizenz (z. B. CC BY 4.0) kenntlich gemacht.

## 4. Entscheidungen für Technologien

Um Skalierbarkeit und leichte Wartbarkeit zu gewährleisten, wurde eine strikte Trennung von Frontend und Backend gewählt. Das systematische Abfragen läuft ausschließlich auf dem Server, was die Endgeräte der Einsatzkräfte entlastet.

- Backend (Python, FastAPI, Uvicorn, WebSockets): FastAPI und der Server Uvicorn bilden einen modernen Industriestandard für asynchrone Anwendungen. Über WebSockets werden neue Datenpakete per Push-Verfahren sofort an die Clients geschickt, statt jedes Gerät permanent per HTTP-Polling abfragen zu müssen. Zusätzlich kann das Backend das kompilierte Frontend selbst ausliefern – so benötigt die gehostete Variante nur einen einzigen Prozess.
- Frontend (React, Vite, Tailwind CSS, Recharts): React wurde wegen der komponentenbasierten Architektur und der hohen Marktdurchdringung gewählt. Das erleichtert das Onboarding neuer Entwickler und das Hinzufügen weiterer Fach-Widgets erheblich.

## 5. Details zur Benutzerschnittstelle

Die Oberfläche ist als Single-Page-Application aufgebaut und skaliert sich automatisch an die Bildschirmgröße an, sodass alle Fachbereiche auf einer Seite sichtbar bleiben. Ein umschaltbarer Hell- und Dunkelmodus deckt den Tagetrieb (maximale Kontraste) und schlecht beleuchtete Räume ab. Die Benutzeroberfläche ist interaktiv: Prognosedetails lassen sich aufklappen, die stündliche Vorschau über Pfeile durchblättern, und jedes Widget verfügt über einen manuellen Refresh-Button mit visuellem Ergebnis-Feedback. Beim Pegel-Widget lassen sich Messstellen zusätzlich suchen und anhand der Referenzwerte (z. B. „über MHW“) filtern, und der Trend-Zeitraum ist per Schalter zwischen 1, 3, 6, 12 und 24 Stunden wählbar (Standard 3 h). Im Wetter-Widget werden die offiziellen DWD-Warnstufen in den Originalfarben des DWD angezeigt. Bei einem Ausfall einer Schnittstelle fängt die Ansicht den Fehler ab und zeigt einen dezenten Hinweis an.